BRIEF TÉCNICO: A LINGUAGEM KOTLIN

Disciplina: Linguagens de Programação | Entrega: Aula 06

---

1. AVALIAÇÃO PELOS CRITÉRIOS DE PROJETO (AULA 01)

1.1 Legibilidade

- Sintaxe Limpa: Elimina semicolons obrigatórios e reduz o código boilerplate (ex.: data class gera equals, hashCode, toString e copy automaticamente).
- Null Safety Visível: A distinção entre tipos anuláveis (String?) e não-anuláveis (String), somada aos operadores ?. e ?: (Elvis), explicita o tratamento de nulos diretamente no código fonte.

1.2 Redigibilidade

- Smart Casts: O compilador infere e converte tipos automaticamente após checagens condicionais (ex.: if (obj is String)), dispensando casts explícitos.
- Funções de Extensão: Permitem estender classes sem utilizar herança ou padrões decoradores complexos.

1.3 Confiabilidade

- Eliminação de NPE: O tratamento de nulidade em tempo de compilação previne exceções de ponteiro nulo em tempo de execução.
- Imutabilidade e Concorrência: Incentivo ao uso de imutabilidade por padrão (val, listOf) e Structured Concurrency via Corrotinas para controle do ciclo de vida assíncrono.

1.4 Custo

- Compilação e Manutenção: O compilador kotlinc é ligeiramente mais lento que o Java puro devido às análises de nulidade e inferência. Em contrapartida, diminui drasticamente o custo de manutenção e depuração. Interopera com 0% de sobrecarga na JVM.

---

2. NOMES, ESCOPOS E TEMPO DE VIDA (AULA 04)

2.1 Tipos de Escopo

1. Escopo Léxico/Estático: Resolução de nomes feita inteiramente em tempo de compilação.
2. Escopo de Bloco: Variáveis em { ... } possuem visibilidade limitada ao bloco.
3. Escopo de Receiver: Funções com receiver (String.() → Unit) injetam this no escopo do bloco.

2.2 Closures e Captura de Variáveis

- Lambdas capturam variáveis do escopo externo (outer scope).
- Diferente do Java, Kotlin permite capturar e mutar variáveis locais (var). O compilador encapsula primitivos mutáveis em wrappers Ref<T> alocados no Heap.

2.3 Gerenciamento de Memória

- JVM Heap & Stack: Primitivos e quadros de execução ficam na Stack; objetos e referências ficam no Heap (gerenciados via Garbage Collector).
- Inline Functions: O uso da palavra-chave inline expande a função no ponto de chamada, eliminando a alocação de objetos de closure no Heap.

---

3. SISTEMA DE TIPOS (AULA 05)

3.1 Posição nos Eixos

- Estático e Forte: Validação de tipos em tempo de compilação. Operações inválidas ou conversões implícitas não são permitidas.

3.2 Equivalência e Inferência

- Equivalência Nominal: Tipos com estruturas idênticas, mas nomes diferentes, não são equivalentes.
- Inferência Local e por Fluxo: O compilador deduz tipos na declaração de variáveis e ajusta tipos via Smart Casts durante a análise de fluxo.

3.3 Coerções

- Sem Widening Implícito: Não há conversão automática entre números (ex.: Int não vira Double). Conversões exigem métodos explícitos (.toDouble()).

---

4. EXEMPLOS EXECUTÁVEIS E GUIA

Como Executar: kotlinc Exemplo.kt -include-runtime -d Exemplo.jar && java -jar Exemplo.jar (ou via kotlinlang.org).

-

```
Kotlin
// Exemplo 1: Null Safety & Smart Cast
fun processa(texto: String?): Int {
    val t = texto ?: return 0 // Elvis operator
    return t.length // Smart cast para String não-anulável
}

// Exemplo 2: Closure Mutável & Inline
inline fun rodar(vezes: Int, bloco: () -> Unit) {
```

```
    for (i in 1..vezes) bloco()
}
fun demoClosure() {
    var c = 0
    rodar(3) { c += 1 } // Captura e altera 'c' sem overhead no Heap
}

// Exemplo 3: Tipagem Nominal & Sem Coerção Implícita
data class UserId(val id: Int)
data class ProdId(val id: Int)
fun demoTipos() {
    val i: Int = 10
    val d: Double = i.toDouble() // Exige conversão explícita
}
```